

THREADLEARN: KẾT HỢP PHÂN TÍCH TĨNH VÀ TINH CHỈNH MÔ HÌNH NGÔN NGỮ LỚN ĐỂ SỬA LỖI ĐỒNG THỜI TRONG JAVASCRIPT

Dồ án môn học — Đại học FPT

Lê Trí Trung Hà Văn Ân

Đại học FPT

letritrung2605@gmail.com

Tháng 6, 2026

Tóm tắt nội dung

Lỗi đồng thời (concurrency bug) trong mã nguồn JavaScript bất đồng bộ thường gây ra các sự cố nghiêm trọng trong môi trường thực tế như mất dữ liệu, treo chương trình hoặc kết quả sai. Các công cụ phát hiện lỗi hiện tại có hai điểm yếu chính: công cụ phân tích tĩnh tìm được lỗi nhưng không thể đề xuất cách sửa, còn mô hình ngôn ngữ lớn (LLM) có thể lập luận về mã nguồn nhưng không chỉ ra được dòng code cụ thể gây lỗi và đòi hỏi mô hình rất lớn (7B–32B tham số). ThreadLearn giải quyết cả hai hạn chế này bằng cách kết hợp bộ phát hiện lỗi tĩnh 5 mẫu với mô hình ngôn ngữ nhỏ Qwen2.5-Coder-1.5B được tinh chỉnh bằng QLoRA ($r=16$, nén 4-bit NF4) trên 783 ví dụ huấn luyện có giải thích từng bước. Tại thời điểm suy luận, hệ thống tìm kiếm BM25 truy xuất 3 tài liệu liên quan nhất từ cơ sở tri thức 2050 tài liệu để bổ sung vào prompt. Trên bộ kiểm thử 20 lỗi đồng thời JavaScript thuộc 8 loại, ThreadLearn với RAG đạt **tỷ lệ vượt qua 75% (15/20)**, so với 40% của mô hình gốc chưa tinh chỉnh và 30% của GPT-3.5-turbo—cải thiện +35 điểm phần trăm.

Từ khóa: lỗi đồng thời, race condition, phân tích tĩnh, tinh chỉnh LLM, Chain-of-Thought, JavaScript

1 Giới thiệu

1.1 Bối cảnh và vấn đề

JavaScript bất đồng bộ (Node.js) được sử dụng rộng rãi trong các hệ thống backend hiện đại, nhưng ngôn ngữ này dễ dẫn đến lỗi khi nhiều thao tác chạy cùng lúc. Một nghiên cứu lớn (NodeCB) đã phân tích 57 lỗi đồng thời thực tế trong 53 dự án Node.js mã nguồn mở và phát hiện: 65% là vi phạm tính nguyên tử, 30% là vi phạm thứ tự thực thi, và 93% gây ra sự cố nghiêm trọng như crash, sai dữ liệu, hoặc treo hệ thống.

Dù vậy, các công cụ hiện tại vẫn còn nhiều hạn chế: Công cụ phân tích tĩnh chỉ phát hiện được lỗi mà không thể gợi ý sửa, và thường sinh nhiều cảnh báo sai. Các công cụ dùng LLM có thể lập luận toàn diện hơn nhưng không xác định được dòng gây lỗi và cần mô hình rất lớn để đạt hiệu quả tốt.

1.2 Khoảng trống trong nghiên cứu hiện tại

Bảng 1 tóm tắt 4 khoảng trống mà ThreadLearn hướng đến giải quyết.

Bảng 1: Các khoảng trống trong công cụ phát hiện và sửa lỗi đồng thời hiện tại.

Ký hiệu	Khoảng trống
G1	Công cụ phân tích tĩnh chỉ phát hiện lỗi, không thể tự động sửa
G2	PCWMs đạt độ chính xác 75,2% nhưng bỏ qua thông tin từ phân tích tĩnh và cần 27.000 mẫu huấn luyện với mô hình lớn
G3	Chưa có hệ thống nào kết hợp đầu ra phân tích tĩnh làm tín hiệu định hướng cho LLM
G4	Chưa có pipeline đầu cuối từ phát hiện → giải thích → sửa lỗi cho JavaScript (Node.js)

1.3 Đóng góp chính

Bài báo này đóng góp 4 thành phần:

1. **race_detector**: bộ phân tích tĩnh 5 mẫu cho JavaScript, tích hợp với bộ tiền xử lý AST.
2. **Tập dữ liệu Hindsight CoT**: corpus huấn luyện 783 ví dụ dạng (code, kết quả detector, lập luận từng bước, bản sửa).

3. **Pipeline ThreadLearn:** hệ thống đầu cuối phát hiện → định dạng → truy xuất tài liệu → sinh bản sửa, phục vụ qua FastAPI server.
4. **Đánh giá thực nghiệm:** so sánh với NodeCB và PCWMs về tỷ lệ sửa đúng, tỷ lệ cảnh báo sai, và độ trễ.

2 Nền tảng lý thuyết

2.1 Phân loại lỗi đồng thời

Bảng 2 tóm tắt các loại lỗi đồng thời phổ biến trong Node.js.

Bảng 2: Phân loại lỗi đồng thời (theo nghiên cứu NodeCB) và ví dụ trong Node.js.

Loại lỗi	Tỷ lệ	Ví dụ trong Node.js
Vi phạm tính nguyên tử	65%	Hai lệnh gọi CSDL chồng chéo gây mất dữ liệu cập nhật
Vi phạm thứ tự thực thi	30%	Upload hoàn thành trước khi callback tạo file chạy
Thiếu tài nguyên	5%	<code>setImmediate</code> bị chặn bởi hàng đợi I/O
Closure bắt sai biến	JS	<code>var i</code> trong vòng lặp + <code>setTimeout</code> : mọi callback đều in giá trị cuối cùng

2.2 Hạn chế của các phương pháp hiện tại

Bảng 3: Hạn chế của các công cụ hiện tại và nhận xét từ nghiên cứu này.

Phương pháp	Hạn chế / Nhận xét
NodeCB	Dùng regex; tỷ lệ cảnh báo sai cao; không sinh bản sửa
PCWMs	Độ chính xác 75,2%; bỏ qua cấu trúc tĩnh; cần mô hình lớn
N1	Đầu ra detector (tên mẫu, phạm vi dòng) cung cấp bằng chứng cụ thể giúp LLM lập luận chính xác hơn
N2	Mô hình nhỏ (1,5B tham số) được tinh chỉnh với lập luận từng bước có nền tảng có thể vượt mô hình lớn dùng prompt chung

2.3 Nền tảng xử lý AST

ThreadLearn dùng thư viện **esprima** (Python thuần) để phân tích cú pháp JavaScript thành cây cú pháp trừu tượng (AST). AST biểu diễn mã nguồn dưới dạng cây có cấu trúc, trong đó mỗi nút là một phần tử ngôn ngữ (hàm, biến, toán tử), giúp trích xuất thông tin chính xác hơn so với xử lý văn bản thông thường.

Bảng 4: Các hàm của bộ tiền xử lý AST và công dụng.

Hàm	Mục đích	Ghi chú
<code>stripComments</code>	Xóa chú thích khỏi mã nguồn	esprima xóa theo phạm vi byte (<code>ast.unparse</code> (Python))
<code>normalizeWhitespace</code>	Chuẩn hóa khoảng trắng và dòng trống	Dựa trên luồng token esprima (JavaScript)
<code>extractFunctions</code>	Liệt kê tất cả hàm trong file	Bắt được cả hàm mũi tên và hàm danh
<code>extract_keywords</code>	Trích xuất 20 tên định danh hàng đầu	Dùng token <code>Identifier</code> của esprima; đầu vào cho BM25

3 Thiết kế hệ thống

3.1 Kiến trúc tổng quan

ThreadLearn gồm 5 module độc lập (Bảng 5). Mỗi module có một nhiệm vụ rõ ràng; chúng giao tiếp qua cấu trúc dữ liệu đơn giản nên có thể thay thế từng phần mà không ảnh hưởng toàn hệ thống.

Bảng 5: Các module của ThreadLearn và vai trò.

Module	Vai trò
<code>ast_preprocessor</code>	Làm sạch và chuẩn hóa mã nguồn đầu vào
<code>race_detector</code>	Phát hiện 5 mẫu race condition trong JavaScript
<code>report_formatter</code>	Chuyển tên mẫu thành mức độ nghiêm trọng và gợi ý sửa
<code>rag_pipeline</code>	Tìm kiếm tài liệu liên quan, bổ sung vào prompt LLM
LLM đã tinh chỉnh	Sinh mã nguồn đã sửa kèm lập luận từng bước
FastAPI server	Cung cấp endpoint HTTP <code>/analyze</code>

3.2 Bộ phát hiện Race Condition tĩnh

Bộ phát hiện quét file mã nguồn theo thứ tự tuyến tính $O(n)$ và xuất báo cáo dạng cấu trúc: `{pattern_id, line_range, description}`. Bảng 6 liệt kê đầy đủ 5 mẫu được phát hiện.

Bảng 6: 5 mẫu lỗi đồng thời mà ThreadLearn phát hiện (chỉ JavaScript).

Nhóm	Mã mẫu	Mô tả
Closure/async	<code>closure_loop_var</code>	Biến vòng lặp bị bắt sai trong closure
	<code>shared_var_settimeout</code>	Biến dùng chung bị thay đổi trong <code>setTimeout</code>
	<code>promise_no_await</code>	Promise được tạo nhưng không được <code>await</code>
Dữ liệu dùng chung	<code>concurrent_write_array</code>	Nhiều callback cùng ghi vào một mảng
	<code>counter_no_atomic</code>	Bộ đếm tăng/giảm không có bảo vệ nguyên tử

Listing 1: Lỗi: `var i` được dùng chung bởi tất cả callback.
Ví dụ minh họa: Biến vòng lặp bị bắt sai.

```

1 for (var i = 0; i < 5; i++) {
2   setTimeout(function() {
3     console.log(i); // luôn in ra 5
4   }, 100);
5 }
```

Listing 2: Sửa: `let i` tạo bản sao riêng cho mỗi vòng lặp.

```

1 for (let i = 0; i < 5; i++) {
2   setTimeout(function() {
3     console.log(i); // in ra 0, 1, 2, 3, 4
4   }, 100);
5 }
```

3.3 Tinh chỉnh mô hình với Hindsight Chain-of-Thought

Mỗi ví dụ huấn luyện có dạng:

(code, kết quả detector, lập luận từng bước, bản sửa)

trong đó phần lập luận được viết *sau khi đã biết bản sửa đúng* (“hindsight”). Viết lập luận theo chiều ngược — bắt đầu từ đáp án và giải thích tại sao đúng — tạo ra tín hiệu huấn luyện nhất quán và sạch hơn.

Bảng 7: Cấu hình tinh chỉnh mô hình.

Tham số	Giá trị
Mô hình gốc	Qwen2.5-Coder-1.5B
Phương pháp	QLoRA: $r=16$, $\alpha=32$, nén trọng số 4-bit NF4
Framework huấn luyện	SFTTrainer (trl $\geq 1.5.0$)
Tín hiệu định hướng	pattern_id + line_range từ detector
Số mẫu huấn luyện	783 ví dụ JSONL
<i>Các lựa chọn đã bác bỏ</i>	
Tinh chỉnh toàn bộ	Chi phí tính toán quá cao
Chỉ dùng RAG	Tìm được tài liệu nhưng không thể sinh bản sửa

3.4 Pipeline RAG tăng cường

1. **Trích xuất từ khóa:** `extract_keywords()` phân tích JavaScript bằng `esprima`, duyệt cây AST và thu thập các token `Identifier` (tên hàm, tên biến, lời gọi API), bỏ qua từ khóa dành riêng của JS. Tên CamelCase như `appendFile` hay `setTimeout` được giữ nguyên, không bị tách thành các từ rời.
2. **Truy xuất tài liệu:** BM25 tính điểm truy vấn với 2.050 tài liệu trong cơ sở tri thức và trả về 3 tài liệu liên quan nhất. Vì tên API được giữ nguyên, các từ trong truy vấn khớp chính xác với từ trong tài liệu, cho kết quả tìm kiếm chính xác hơn tokenizer văn bản thông thường.
3. **Định dạng:** `report_formatter` chuyển `pattern_id` thành nhãn mức độ nghiêm trọng và gợi ý sửa ngắn gọn.
4. **Sinh mã:** LLM đã tinh chỉnh nhận 3 tài liệu truy xuất được thêm vào đầu prompt (đúng định dạng completion như lúc huấn luyện) và xuất mã đã sửa.

Tại sao trích xuất từ khóa bằng AST tốt hơn regex? Tokenizer regex tách `setTimeout` thành `["set", "timeout"]` và `appendFile` thành `["append", "file"]`, tạo ra truy vấn chứa các từ quá chung chung. `esprima` đọc cây cú pháp và trả về token định danh nguyên vẹn, nên truy vấn khớp chính xác với tên API trong cơ sở tri thức — dẫn đến truy xuất chính xác hơn cho các mẫu đồng thời đặc thù của JavaScript.

Bảng 8: Tóm tắt cài đặt kỹ thuật.

Hạng mục	Chi tiết
Ngôn ngữ / framework	Python 3.10+, FastAPI
Tổng số dòng code	≈ 1.200 dòng
<code>race_detector.py</code>	320 dòng, 10 bộ phát hiện, quét tuyến tính
File adapter mô hình	LoRA SafeTensors tại <code>./threadlearn-qwen2.5-coder-1.5b/final_adapter</code>
Cơ sở hạ tầng	Docker: Redis cache (localhost:6379), Atlas MongoDB
Kiểm soát đồng thời	Semaphore trong FastAPI (mỗi lúc 1 yêu cầu LLM)
Xử lý lỗi	Hệ thống vẫn chạy nếu <code>ast_preprocessor</code> không import được

4 Cài đặt hệ thống

5 Đánh giá thực nghiệm

5.1 Thiết lập thực nghiệm

Bảng 9: Thiết lập thực nghiệm đánh giá.

Hạng mục	Chi tiết
Baseline 1	Qwen2.5-Coder-1.5B chưa tinh chỉnh, cùng định dạng prompt
Baseline 2	GPT-3.5-turbo không có ví dụ, temperature 0, API OpenAI
Hệ thống của chúng tôi	ThreadLearn: mô hình đã tinh chỉnh + truy xuất BM25
Bộ kiểm thử	20 lỗi đồng thời JS tự soạn, 8 loại lỗi
Định dạng prompt	"Convert to concurrent JavaScript:\n\n{code}\n"
Thang điểm	PASS = có mẫu sửa đúng; PARTIAL = có code nhưng sai mẫu; FAIL = không có
Phần cứng	NVIDIA RTX 4060 Laptop GPU, float16
Thời điểm đánh giá	11/06/2026

5.2 Kết quả so sánh tổng thể

Bảng 10 so sánh tỷ lệ vượt qua trên bộ kiểm thử 20 trường hợp.

Bảng 10: Kết quả sửa lỗi trên bộ kiểm thử 20 trường hợp JavaScript (11/06/2026).

Phương pháp	Pass	Partial	Fail	Tỷ lệ
GPT-3.5-turbo (không có ví dụ)	6	11	3	30%
Qwen2.5-Coder-1.5B (chưa tinh chỉnh)	8	9	3	40%
ThreadLearn RAW (tinh chỉnh, không RAG)	14	6	0	70%
ThreadLearn + RAG (của chúng tôi)	15	5	0	75%

Ba quan sát nổi bật:

1. Tinh chỉnh mô hình đơn thuần cho +30 điểm phần trăm so với mô hình gốc (70% vs 40%), và kết quả này phụ thuộc hoàn toàn vào việc dùng đúng định dạng prompt huấn luyện. Một lần chạy thử sai dùng chat template chỉ đạt 35%.
2. Thêm RAG tăng thêm +5 điểm phần trăm (75% vs 70%) bằng cách cung cấp tài liệu liên quan cho các trường hợp khó (ví dụ: mẫu upsert cho test_03).
3. ThreadLearn không có trường hợp FAIL nào ở cả hai chế độ — luôn sinh ra mã nguồn — trong khi GPT-3.5 và mô hình gốc đều có 3 trường hợp FAIL.

5.3 Kết quả theo từng loại lỗi

Bảng 11: Tỷ lệ sửa đúng của ThreadLearn+RAG theo từng loại lỗi (20 trường hợp).

Loại lỗi	Mẫu sửa dự kiến	Pass/Tổng	Tỷ lệ
Race Condition	async/await, appendFile, upsert	3/5	60%
Chặn Event Loop	yield generator, fs.promises, renderAsync	5/5	100%
Unhandled Rejection	try/catch + next(error)	1/1	100%
Double Callback	return trước callback thứ hai	0/1	0%
Zalgo	process.nextTick để đồng nhất async	0/1	0%
Mất ngữ cảnh <code>this</code>	Dùng hàm mũi tên	1/1	100%
Callback Hell	Viết lại bằng async/await	1/1	100%
Cạn kiệt tài nguyên	Giới hạn đồng thời bằng slice	1/1	100%
Await tuần tự	Chạy song song bằng Promise.all	1/1	100%
Thiếu Promise.all	Thêm Promise.all	1/1	100%
Buffer Leak	Thêm stream error handler	0/1	0%
Thứ tự Event Loop	Promise.resolve().then	1/1	100%
Tổng cộng		15/20	75%

ThreadLearn đạt 100% trên 8 trong 12 loại lỗi. Ba loại còn thất bại — Zalgo, Double Callback, Buffer Leak — là các mẫu hiếm và đặc thù mà mô hình 1,5B chưa khái quát được từ tập huấn luyện.

5.4 Phân tích đóng góp từng thành phần

Yếu tố lớn nhất ảnh hưởng đến kết quả là định dạng prompt. Dùng chat template thay vì định dạng completion đúng của lúc huấn luyện làm tụt tỷ lệ từ 70% xuống 35% — giảm 35 điểm phần trăm chỉ do lỗi định dạng, che giấu phần lớn lợi ích của tinh chỉnh trong lần chạy đánh giá đầu tiên.

Bảng 12: Đóng góp của từng thành phần vào kết quả cuối (bộ kiểm thử 20 trường hợp).

Cấu hình	Pass/20	Tỷ lệ
GPT-3.5-turbo (không tinh chỉnh, không RAG)	6	30%
Qwen2.5-Coder-1.5B gốc (không tinh chỉnh, không RAG)	8	40%
ThreadLearn RAW (chỉ tinh chỉnh, không RAG)	14	70%
ThreadLearn + RAG (tinh chỉnh + truy xuất tài liệu)	15	75%
<i>Đóng góp của tinh chỉnh</i>	+30 điểm phần trăm so với mô hình gốc	
<i>Đóng góp của RAG</i>	+5 điểm phần trăm so với chỉ tinh chỉnh	
<i>Sai định dạng prompt</i>	35% (chat template) vs 70% (định dạng)	

Bảng 13: Thông lượng và độ trễ phản hồi (LLM giả lập, 10 client song song, N=200 yêu cầu).

Chỉ số	Giá trị
Thông lượng	86 yêu cầu/giây
Độ trễ trung vị (P50)	3,2 ms
Phân vị thứ 95 (P95)	112 ms
Phân vị thứ 99 (P99)	129 ms
Lý do P95 cao	Khi 10 client gửi yêu cầu cùng lúc, chúng xếp hàng chờ semaphore LLM; P95 tăng từ 8 ms (1 client) lên 112 ms (10 client)

5.5 Hiệu năng và độ trễ

6 Nghiên cứu liên quan

Bảng so sánh tính năng.

7 Kết luận

Lỗi đồng thời vẫn là một trong những nguyên nhân chính gây sự cố trong các ứng dụng JavaScript bất đồng bộ, nhưng chưa có công cụ nào vừa phát hiện vừa sửa lỗi một cách đáng tin cậy ở quy mô mô hình nhỏ. ThreadLearn kết hợp bộ phát hiện lỗi tĩnh 5 mẫu với mô hình ngôn ngữ nhỏ (Qwen2.5-Coder-1.5B) được tinh chỉnh trên các ví dụ lập luận từng bước có nền tảng và pipeline truy xuất tài liệu BM25, tạo ra hệ thống sửa lỗi đầu cuối cho JavaScript (Node.js). Trên bộ kiểm thử 20 trường hợp thuộc 8 loại lỗi, ThreadLearn với RAG đạt **tỷ lệ vượt qua 75% (15/20)**, so với 40% của mô hình chưa tinh chỉnh và 30% của GPT-3.5-turbo — cải thiện +35 điểm phần trăm, cho thấy định dạng huấn luyện đúng và tinh chỉnh chuyên biệt theo domain có thể cải thiện đáng kể chất lượng sửa lỗi ngay cả với mô hình chỉ 1,5B tham số.

Bảng 14: So sánh ThreadLearn với các công trình liên quan.

Nhóm	Công trình tiêu biểu	Điểm khác biệt
Phát hiện race condition tĩnh	ThreadSanitizer, Helgrind, ESLint async	Chỉ phát hiện, không sửa; không dùng LLM
Nghiên cứu thực nghiệm lỗi	NodeCB, lỗi đồng thời Android	Cung cấp phân loại lỗi; không sửa tự động
LLM phân tích mã nguồn	PCWMs, CodeBERT, CodeT5	Chỉ dùng LLM; bỏ qua cấu trúc tĩnh; cần mô hình lớn
Tinh chỉnh cho code	QLoRA, SFT on code, PEFT	Cùng kỹ thuật huấn luyện; không nhắm đến race condition

Bảng 15: Công cụ nào hỗ trợ phát hiện, sửa lỗi, JavaScript, và tinh chỉnh.

Công cụ	Phát hiện	Sửa lỗi	JS	Tinh chỉnh
ThreadSanitizer		×	×	×
ESLint async		×		×
NodeCB (quy tắc)		×		×
PCWMs (chỉ LLM)				
ThreadLearn				

Hướng phát triển tiếp theo. Mở rộng sang AST TypeScript (thay thế fallback regex hiện tại); bổ sung mẫu MPI/CUDA theo hướng của PCWMs.

Tài liệu

- [1] T. Xu và cộng sự. NodeCB: Nghiên cứu lỗi đồng thời trong Node.js. *Kỷ yếu IEEE/ACM ASE*, trang 87–98, 2017.
- [2] G. Singh và cộng sự. PCWMs: Prompting với Chain-of-Thought World Models để phát hiện lỗi đồng thời. *arXiv:2604.20926v2*, 2026.
- [3] T. Detrmers và cộng sự. QLoRA: Tinh chỉnh hiệu quả các LLM được lượng hóa. *Kỷ yếu NeurIPS*, 2023.
- [4] Nhóm Qwen. Báo cáo kỹ thuật Qwen2.5-Coder. *arXiv:2409.12186*, 2024.